

Alpha4'

V1.0

Objetivo: Implementar agentes de Inteligência Artificial para jogar *Connect N* (4 em Linha) utilizando técnicas de procura adversarial e procura baseada em simulação.

Neste trabalho, será fornecido um motor de jogo completo e uma interface gráfica (GUI). O objetivo é desenvolver agentes que tomem decisões automaticamente, respeitando uma API pré-definida.

Descrição do Jogo

O *Connect N* é um jogo de dois jogadores, por turnos, onde:

- Os jogadores colocam peças numa coluna do tabuleiro
- As peças caem para a posição mais baixa disponível
- O objetivo é alinhar **N peças consecutivas** (horizontal, vertical ou diagonal)
- O jogo termina quando:
 - Um jogador vence, ou
 - O tabuleiro fica cheio (empate)

Os parâmetros do jogo (linhas, colunas e valor de N) são configuráveis.



Figura 1 - jogo do 4 em linha

Estrutura do Projeto

O projeto está organizado em 6 ficheiros com as seguintes componentes:

Lógica do jogo

- Classe Connect4Board:
 - Representa o estado do tabuleiro;
 - Não deve ser modificada;

Jogadores

- Player (classe base);
- HumanPlayer;
- RandomAIPlayer;
- MinimaxAIPlayer (a implementar);
- MCTSAIPlayer (a implementar);

Interface gráfica (class Connect4Gui)

- Implementada com Pygame;

Loop principal (class Connect4Game)

- Função run_game(player1, player2, headless=False, rows = 6, cols = 7)

API do Jogo

Os agentes devem interagir com o jogo usando apenas os seguintes métodos implementados na classe Connect4Board:

- `board.get_valid_moves()`
- `board.drop_piece(col, piece)`
- `board.check_winner(piece)`
- `board.is_board_full()`
- `board.copy()`
- `board.grid` (leitura do estado)

O tabuleiro deve ser tratado como **read-only**.

Para simular jogadas, deve ser utilizado:

```
new_board = board.copy()
```

Interface dos Agentes

Todos os agentes devem ser implementados como subclasses de:

```
class Player:
    def get_move(self, board):
        pass:
```

Cada agente deve implementar o método `get_move`:

- Recebe o estado atual (board);

- Retorna uma jogada válida, i.e. um inteiro que corresponde à coluna onde deve ser feita a jogada;

Agente 1: Minimax com Alpha-Beta

Criar uma classe, por exemplo:

```
class MinimaxAIPlayer(Player):
```

Deve ser implementado um agente com:

- Algoritmo **Minimax**
- **Poda Alpha-Beta**
- Profundidade máxima fixa (max_depth)

A pesquisa termina quando:

- Um estado terminal é atingido, ou
- A profundidade máxima é alcançada.

Função Heurística

Deve ser implementada uma função na classe `MinimaxAIPlayer` com a seguinte assinatura:

```
evaluate_board(game, player)
```

Esta função deve:

- Valorizar estados vencedores;
- Penalizar estados perdedores;
- Considerar:
 - Sequências de 2 e 3 peças;
 - Ameaças do adversário;
 - Posição no tabuleiro (ex: centro);

Agente 2: Monte Carlo Tree Search (MCTS)

Criar uma classe, por exemplo:

```
class MCTSAIPlayer(Player):
```

Deve ser implementado MCTS de raiz, incluindo:

- Seleção (UCB1);
- Expansão;
- Simulação (jogo aleatório);
- Retropropagação;

Iterações

O algoritmo deve funcionar com um número fixo de iterações:

```
max_iterations
```

Cada iteração inclui todas as fases do MCTS.

Retropropagação

- Vitória → crédito ao jogador vencedor;
- Empate → valor neutro;

Implementação

Adicionar ao código:

- Classe `MinimaxAIPlayer`
- Classe `MCTSAIPlayer`
- Função heurística

Sem alterar:

- `Connect4Board`
- `Connect4Game.run_game`
- `Connect4Gui`
- `Player`

Execução

Exemplo:

```
p1 = MinimaxAIPlayer(piece=1)
p2 = MCTSAIPlayer(piece=2)
game = Connect4Game()
game.run_game(p1, p2, headless= False)
```

Comparação de Algoritmos

Para garantir uma comparação justa entre os diferentes algoritmos, os tempos de execução devem ser equivalentes. Para tal, é necessário ajustar adequadamente tanto o parâmetro *max_depth* como o parâmetro *max_iterations*.

Devem ser apresentados resultados para três combinações distintas destes parâmetros, assegurando sempre que ambos os algoritmos apresentam tempos de execução sensivelmente idênticos.

Devem ser realizados testes entre:

- Minimax vs Aleatório;
- MCTS vs Aleatório;
- Minimax vs MCTS;
- (Opcional) Humano vs IA;

Devem ser analisados:

- Taxa de vitórias;
- Duração dos jogos;
- Diferenças de comportamento;

Deve preencher a seguinte tabela num ficheiro excel chamado resultados.xlsx.

Comparação	Nº de Jogos	Vitórias Jogador 1	Vitórias Jogador 2	Empates	Taxa de Vitórias Jogador 1	Taxa de Vitórias Jogador 2	Duração Média do Jogo	Duração Máxima	Duração Mínima	Diferenças de Comportamento Observadas
Minimax vs Aleatório										
MCTS vs Aleatório										
1º comb Minimax vs MCTS										
2º comb Minimax vs MCTS										
3º comb Minimax vs MCTS										
Humano vs IA (Opcional)										

Entrega:

O trabalho tem de ser realizado em grupos de 2. Tem de ser entregue um ficheiro zip com todos os ficheiros deste projeto, até às 23:59 do dia 25/05/2026 no Moodle.

O ficheiro zip tem de possuir como nome Nome1_Número1_Nome2_Número2.zip (por exemplo: MariaSilva_123456_JoaoLopes_654321.zip). É obrigatório manter os nomes dos ficheiros e métodos pedidos para a implementação pois serão realizados testes automáticos.